One parameter governs the Boussinesq–compressible gap: the break-symmetry axis $\varepsilon = \Delta T/T_0$ and a physics-prior surrogate

Author(s)
Affiliation(s)

October 2026

Abstract

Surrogates of compressible natural convection are routinely trained on, or validated against, Boussinesq solutions. We show that for two-dimensional Rayleigh–Bénard convection the gap between the two physics is controlled by **one exactly identifiable parameter**, the relative temperature contrast $\varepsilon = \Delta T/T_0$, and we turn that fact into a design rule for surrogates.

1. ε **is the only break-symmetry parameter.** In the non-dimensionalisation used here the Froude number is not free ($Fr^2 = \beta\Delta T = \Delta T/T_0$ identically) and the Mach-number effect is $O(Ma^2)$, unmeasurable here (changing Ma by a factor 24 changes the correction by less than one part in 10^5). The break-symmetry group is one-dimensional.
2. **The law.** Over 600 random-boundary-condition samples plus 27 independent small-ε checks ($\varepsilon = 0.02$–1.9, two decades), the relative correction obeys $R = \|Y - X\|/\|Y\| = A\varepsilon^p$ with $p = 1.023 \pm 0.018$ (u_x), 0.995 ± 0.019 (u_y), 1.024 ± 0.077 (θ) and 1.009 ± 0.015 (p), and $A = 0.147, 0.193, 0.010, 0.148$; the intercept vanishes ($+0.0003 \pm 0.0047$ for u_x, i.e. 0.06σ). The exponent is window-averaged: over the full two decades it is 1.02, and 1.23–1.32 if the same data are fitted from $\varepsilon = 0.8$ upwards, which is the fingerprint of the positive ε^2 term ($R = A\varepsilon(1 + \kappa\varepsilon)$, $\kappa = 0.42$–0.83, Sec. 5.1). *Inside the random-wall family*, 134 wall-mode families add only 2–3 percentage points of adjusted R^2 to $u_x/u_y/p$ (8.5 pp to θ, Sec. 5.1); across families the prefactor does move (Sec. 5.7). A weak Rayleigh-number trend in the prefactor is real and reproducible **in an independent second geometry** for u_y ($q = -0.083 \pm 0.005$ vs -0.077 ± 0.019) and θ ($+0.514 \pm 0.015$ vs $+0.447 \pm 0.045$). A 6.2-fold change in Prandtl number—air to water, 104 strictly paired samples—leaves the exponent unchanged and the prefactor to within 5% on the paired comparison (median paired ratio $R_{\text{water}}/R_{\text{air}} = 1.006/0.999/0.999/0.951$).
3. **Why.** Expanding the compressible member in ε gives an $O(1)$ system that is term-by-term the Boussinesq equations and an $O(\varepsilon)$ source from the density-modulated inertia and the non-solenoidal velocity field, $\nabla\cdot\boldsymbol{u}_1 = \nabla\cdot(\theta_0\boldsymbol{u}_0)$. Because the linearised operator is ε-independent, $D = Y - X = \varepsilon P(X) + \varepsilon^2 Q(X) + \ldots$ with a shape P that does not rotate with ε—confirmed by $\cos(D(\varepsilon), D(1.2)) = 0.95$–$1.00$ and by a direct check of the divergence relation (cosine 0.994 for $\varepsilon < 0.6$). Two terms of the expansion carry 99.0% of the correction energy per configuration.
4. **Consequence for surrogates.** Once ε leaves the value a surrogate was trained at, no model that sees only the Boussinesq field can recover the difference: at the single ε a published data set offers, the exact transfer error is 25% (u_x), 30% (u_y) and 35% (p) at $\varepsilon = 1.9$, and even the best single ε-blind field leaves 22–29% there (Sec. 5.3). Writing the output as $D = \varepsilon P(X)$ and letting the network predict P attacks exactly that: it beats the ε-aware plain architecture by 1.61× in extrapolation and the ε-blind one by 2.44×, and it lets the network find the ε^2 term by itself (learned curvature $\beta = 0.23$–0.42 in all four channels, Sec. 5.5). With ε moved fully outside the network, so that it never enters the input, it is what makes a **single-ε data set reusable**: a surrogate trained on a public 4000-pair data set that exists only at $\varepsilon = 1.2$ is applied, without retraining, to our own $\varepsilon \in [0.2, 1.9]$ data and reaches 0.093 (u_x) at $\varepsilon > 1.3$—better than

the same prior trained on our own data and 3.2× better than doing nothing. Across a different boundary-condition family the same transfer fails (0.29, i.e. no better than doing nothing), because the prefactor is a functional of the flow (Secs. 5.7–5.8).

5. **External validation, five held-out data sets.** The law reproduces on that published data (independent group, independent generator, $\varepsilon \equiv 1.2$): $R = 0.180/0.244/0.023/0.186$ for (u_x, u_y, θ, p) against 0.184/0.225/0.023/0.189 on our own $\varepsilon \approx 1.2$ slice, even the secondary Rayleigh-number correlations match ($r = +0.65$ for θ in both), and every published `relL2` value of all five splits (38 400 values: 9600 samples × 4 channels) reproduces to within 1.2×10^{-7}, i.e. float32 round-off. The prefactor is not universal—between boundary-condition families at fixed ε it moves by 1.66× (u_x), 1.38× (u_y), 4.05× (θ) and 1.07× (p).

The practical statement is narrow and testable: the ε dependence of the Boussinesq–compressible pair is known in closed form, so it should be put outside the network rather than discovered from data. The same output structure transfers unchanged from a U-Net to a Fourier neural operator, and the measured speed-up is 5.4×10^3 per sample and 6.9×10^4 in batches.

Keywords: natural convection; Boussinesq approximation; low-Mach compressible flow; physics-informed surrogate; neural operator; break symmetry

1 Introduction

Steady natural convection is a workhorse of thermal engineering and it is expensive. A converged pair of two-dimensional solves on a 64^2 mesh costs 34 s of wall clock on an eight-way desktop (Sec. 5.11); the three-dimensional problems that engineering actually needs cost hours to days. That is the standard motivation for surrogate modelling, and the standard response is to train a network that maps a cheap model's output to an expensive model's output [1, 2].

For convection the two models are not two arbitrary fidelities: they are the Boussinesq (Oberbeck–Boussinesq) equations and the compressible low-Mach equations. Boussinesq is exact only in the limit of vanishing relative temperature contrast; the corrections are $O(\beta\Delta T)$ and have been characterised asymptotically for decades [3, 4]. Recent work makes the pair directly learnable. Menglik et al. [5] publish paired Boussinesq→compressible steady solutions in two dimensions (4000 training pairs plus held-out splits, CC BY 4.0) and train a Fourier neural operator [6] to predict the compressible field from the Boussinesq field. That data set—which we use as an independent validation set in Secs. 5.7–5.8—contains exactly **one** temperature contrast: $\Delta T = 1.2$ at $T_0 = 1$.

The single-contrast design is the practical crux. A surrogate fitted on such a data set has learned the correction at one operating point. At any other temperature contrast the Boussinesq input already differs from the input seen in training, so the correction the network memorised is the wrong correction—not merely an inaccurate one—and the only remedy on offer is to generate a second paired data set, i.e. to pay again exactly the cost the surrogate was meant to remove. We show in Sec. 5.3 that this is not a modelling failure that more capacity or a better architecture can repair: it is an information barrier, and we measure it.

This paper makes the barrier explicit and then removes it.

What governs the gap. For the non-dimensionalisation used here the Froude number is not free: $Fr^2 = \beta\Delta T = \Delta T/T_0$ identically, and the Mach number enters the equations only at $O(Ma^2)$ (Sec. 2.2, and measured: a factor-24 change in Ma moves the correction by less than 10^{-5}). The break-symmetry group is therefore **one-dimensional**, and it has a name: $\varepsilon = \Delta T/T_0$.

What the gap looks like. Expanding the compressible member in ε gives an $O(1)$ system that is term-by-term the Boussinesq system, plus an $O(\varepsilon)$ source produced by the density-modulated inertia and by the non-solenoidal velocity field $\nabla\cdot\boldsymbol{u}_1 = \nabla\cdot(\theta_0\boldsymbol{u}_0)$. Because the linearised operator

is ε-independent, the correction factorises, $Y - X = \varepsilon P(X) + \varepsilon^2 Q(X) + \ldots$, with a shape P that does not rotate as ε moves (Sec. 3). Measured over two decades of ε (627 samples, Sec. 5.1) the relative correction obeys $R = A\varepsilon^p$ with $p = 1.02$ in all four channels (a window-averaged exponent: the same 627 points give 1.23–1.32 when fitted from $\varepsilon = 0.8$ upwards, the ε^2 curvature of Sec. 5.1), a vanishing intercept, and a prefactor whose weak Rayleigh-number trend reproduces in a second geometry and in a third fluid.

What it buys. Because the ε dependence can be written outside the network, a surrogate can predict $P(X)$ and multiply by ε afterwards, keeping ε out of the input. The result is also an accuracy gain in distribution when data is scarce (Sec. 5.6: 1.4× better than an ε-augmented baseline at $n = 50$, crossover near $n \approx 300$). It is a change of regime: a network trained on a public data set that exists at a single ε is applied, without retraining, to our $\varepsilon \in [0.2, 1.9]$ and reaches relL2 = 0.093 (u_x) where the plain model of the same budget reaches 0.144 (Sec. 5.8), the extrapolation error falls by 1.61× relative to the plain architecture (Sec. 5.5), the network discovers the ε^2 curvature of the expansion on its own (learned $\beta = 0.23$–0.42 in all four channels), and the same prior transfers unchanged to a Fourier neural operator.

1.1 Related work

Surrogates for convection. Reduced-order models replace the discretised equations by a small number of modes; parametric dynamic mode decomposition is a representative recent example [7]. Graph convolutional networks exploit the mesh and have been applied to natural convection in annuli [8] and to geometry-adaptive steady convection with physics-informed losses [9]. Operator learning removes the mesh-resolution constraint: DeepONet approximates the solution operator from branch and trunk networks [10], physics-informed DeepONets add the residual to the loss [11], and Fourier neural operators (FNOs) learn a global convolution in Fourier space [6]. FNOs have since been used for sparse-field reconstruction [12], real-time heat-pipe startup prediction [13], and operator-level uncertainty quantification in cooling-channel optimisation [14]. Physics-informed neural networks [15] and their heat-transfer variants [16, 17] take the opposite route: they solve one instance by minimising a PDE residual, rather than learning a family of instances. Reviews of the numerical analysis and of the application landscape are given by De Ryck and Mishra [18], Meng et al. [2] and Luo et al. [19].

Physics priors and structure. A physics prior can enter as a residual penalty, as a conservation law built into the architecture, or as an algebraic factorisation of the output. The first two are well developed; the third is the subject of this paper. Residual-based corrector operators add a learned correction on top of a neural-operator prediction and improve accuracy without retraining the base operator [20, 21]. Physics-informed extrapolation of deep operators has been demonstrated by adding sparse observations or residual terms [22], and transfer operator learning addresses distribution shift between parameter regimes [23]. Multi-fidelity surrogates combine cheap and expensive data through LSTM corrections [24], feature-adjacent physics-informed learning [25] or transfer-learned FNOs [26]. These methods reduce the amount of high-fidelity data, but they still learn the parameter dependence from samples. The present work differs in that the parameter dependence is derived, and only the spatial shape of the correction is learned.

Boussinesq versus compressible convection. The Oberbeck–Boussinesq approximation and its validity limits are classical [3, 4]; the review of Ahlers et al. [27] summarises the heat-transfer and large-scale dynamics of turbulent Rayleigh–Bénard convection. Non-Oberbeck–Boussinesq (NOB) effects have been measured under strong temperature contrast [28], in cryogenic helium [29] and in a water-filled differentially heated cavity [30]. On the numerical side, low-Mach compressible formulations for enclosed thermal convection have been developed and compared with

the Boussinesq limit [31, 32], and the rigorous derivation of the Oberbeck–Boussinesq approximation has recently been revisited [33]. What is missing from this literature is a quantitative, closed-form account of how the *difference between the two solutions* scales with the temperature contrast, and a surrogate design that uses that scaling. The works above either fix ε, treat it as one more input channel, or study NOB corrections at the level of gross heat transfer; none turns the correction into an output factorisation.

Contributions.

1. We identify the exact, one-dimensional break-symmetry parameter of the Boussinesq–compressible pair and show the group is one-dimensional rather than two (Sec. 2.2).
2. We derive the correction's factorisation from an ε-expansion and verify all five of its predictions numerically, including the divergence relation (Sec. 3).
3. We calibrate the resulting law on 627 paired samples spanning two decades of ε, in two geometries and two Prandtl numbers, and validate it on an independently generated public data set—including the secondary Rayleigh-number trends (Secs. 5.1, 5.7, 5.9, 5.12).
4. We turn the law into a surrogate design rule ($D = \varepsilon P(X)$), measure what it is worth ($1.61\times$ in extrapolation), and show it makes an existing single-ε public data set reusable, with the speed-up measured end to end (5.4×10^3 per sample) (Secs. 5.4–5.8, 5.11).

All results are two-dimensional. Section 6 states the measured cost of the three-dimensional problem on the same hardware and gives the concrete reasons we do not claim a 3-D sweep.

2 Problem, discretisation and the break-symmetry parameter

2.1 Two members per sample

Every sample is a *paired* steady solution of one physical configuration with two equation sets on the same mesh ($res = 64$, $L = 1$, $Pr = 0.71$, $\gamma = 1.4$). The high-fidelity member Y is the compressible low-Mach formulation in $\boldsymbol{w} = (\boldsymbol{u}, T, \rho, p)$:

$$\nabla \cdot (\rho \boldsymbol{u}) = 0, \tag{1}$$

$$\nabla \cdot (\rho\, \boldsymbol{u} \otimes \boldsymbol{u}) = -\nabla p + \frac{1}{Re}\nabla \cdot \boldsymbol{\sigma} - \frac{1}{Fr^2}\rho \boldsymbol{g}, \tag{2}$$

$$\nabla \cdot (\rho E \boldsymbol{u}) = \frac{\gamma}{Re\, Pr}\nabla^2 e + \nabla \cdot (\boldsymbol{P}_{\text{work}}\boldsymbol{u}) + O(Ma^2), \tag{3}$$

$$\rho e = \gamma Ma^2 p + 1, \qquad Re = (Ra/Pr)^{1/2}. \tag{4}$$

The low-fidelity member X is the Boussinesq system in $(\boldsymbol{u}, T, p)$, written in the *same* free-fall scales:

$$\nabla \cdot \boldsymbol{u} = 0, \tag{5}$$

$$\boldsymbol{u} \cdot \nabla \boldsymbol{u} = -\nabla p + \left(\frac{Pr}{Ra}\right)^{1/2} \nabla^2 \boldsymbol{u} + \theta \hat{\boldsymbol{g}}, \tag{6}$$

$$\boldsymbol{u} \cdot \nabla T = (Ra\, Pr)^{-1/2}\nabla^2 T, \qquad \theta = \frac{T - T_0}{\Delta T}. \tag{7}$$

Both are solved with Taylor–Hood-type mixed elements, damped Newton iterations and a MUMPS factorisation; steady states are reached by an adaptive march with a convergence guard. The temperature boundary data are random Fourier wall temperatures with per-wall mode dials.

Table 1: The three candidate break-symmetry parameters. Only ε survives as a free axis in the present non-dimensionalisation.

candidate	evidence	conclusion
Fr	the generators state $Fr^2 = \beta\Delta T = \Delta T/T_0$ identically; the released 2-D code pinned $Fr = 1.095$, correct only at $\Delta T = 1.2$	not free; pinned-Fr sweeps of $T_0/\Delta T$ are an artefact
Ma	fixing Ra, T_0, ΔT and changing Ma by 24× leaves the correction amplitude unchanged to five significant figures	$O(Ma^2) \approx 10^{-6}$: unmeasurable
$\varepsilon = \Delta T/T_0$	same boundary-condition seed stream, ΔT swept	the clean, one-dimensional break-symmetry axis

2.2 Fr is not a parameter, Ma is negligible, ε is the axis

2.3 The low-fidelity solution is exactly ε-blind

θ and $Pr\,Ra$ are the only combinations that appear in the Boussinesq weak form; neither T_0 nor ΔT appears alone. Hence, with the thermal data prescribed in θ and Ra fixed, X **cannot depend on** ε. This was verified to solver precision: across 20 configurations × 5 values of ε (same Ra, bit-identical θ boundary stream), the maximum relative deviation of X is 9.77×10^{-8}.

> **Corollary.** The correction $Y - X$ is information that is *absent from the surrogate's input*: no amount of capacity or training data can recover it from X alone. The limit is informational, not architectural (Sec. 5.3).

3 Asymptotic theory: why the correction factorises as $\varepsilon P(X)$

Write $T = T_0(1 + \varepsilon\theta)$ with $\theta = O(1)$. With $\rho = 1/T$ (the equation of state and initial condition used here) and $Fr^2 = \varepsilon$, the buoyancy source is

$$\frac{\rho \boldsymbol{g}}{\varepsilon} = \frac{\boldsymbol{g}}{\varepsilon} - \theta\boldsymbol{g} + \varepsilon\theta^2\boldsymbol{g} + \dots, \tag{8}$$

so the $O(1)$ balances are

$$\nabla \cdot \boldsymbol{u}_0 = 0, \tag{9}$$

$$\boldsymbol{u}_0 \cdot \nabla \boldsymbol{u}_0 = -\nabla\tilde{p}_0 + Re^{-1}\nabla^2\boldsymbol{u}_0 + \theta_0\hat{\boldsymbol{g}}, \tag{10}$$

$$\boldsymbol{u}_0 \cdot \nabla\theta_0 = (Re\,Pr)^{-1}\nabla^2\theta_0, \tag{11}$$

which is **term-by-term the Boussinesq system solved as** X ($Re^{-1} = (Pr/Ra)^{1/2}$). The correction at $O(\varepsilon)$ is driven by the density modulation:

$$\nabla \cdot \boldsymbol{u}_1 = \nabla \cdot (\theta_0\boldsymbol{u}_0), \tag{12}$$

$$\nabla \cdot (\boldsymbol{u}_1 \otimes \boldsymbol{u}_0 + \boldsymbol{u}_0 \otimes \boldsymbol{u}_1 - \theta_0\boldsymbol{u}_0 \otimes \boldsymbol{u}_0) = -\nabla\tilde{p}_1 + Re^{-1}\nabla^2\boldsymbol{u}_1 + \theta_1\hat{\boldsymbol{g}} + \theta_0^2\hat{\boldsymbol{g}}. \tag{13}$$

Since L, the linearisation of the Boussinesq operator about X, does not contain ε,

$$D = Y - X = -\varepsilon L^{-1}S_1[X] - \varepsilon^2 L^{-1}S_2[X] + \dots = \varepsilon P(X) + \varepsilon^2 Q(X) + \dots \tag{14}$$

Five predictions follow, and all five hold:

The drift of the divergence check with ε ($0.994 \to 0.901$) is the truncated ε^2 term, which is exactly what the fifth line measures independently.

Domain of validity. $\varepsilon < 2$, since the cold wall $T_c = T_0 - \Delta T/2$ must be positive.

Table 2: The five testable consequences of the expansion and their measured values.

prediction	measurement
solver data sets	`code/dataset/` contains the data themselves: 600 main (`run_batch.sh`), 100 paired (`run_paired.sh`), 27 $\varepsilon \to 0$ (`run_eps0.sh`), 104 classical-cavity (`run_cavity.sh`), 104 water at $Pr = 4.38$ (`run_water.sh`); `run_cpu_queue.sh` adds the 2:1 geometry and the tight-tolerance reruns
$R \to 0$ as $\varepsilon \to 0$	intercept $+0.0003 \pm 0.0047$ (u_x); 0.06σ from zero
$R \propto \varepsilon$	$p = 1.023 \pm 0.018$ / 0.995 ± 0.019 / 1.009 ± 0.015 ($u_x/u_y/p$), window-averaged over the two decades; the residual is the ε^2 term (Sec. 5.1)
the shape of D does not rotate with ε	$\cos(D(\varepsilon), D(1.2)) = 0.95$–$1.00$, four channels $\times$ five values of ε
$\nabla \cdot \boldsymbol{u}_1 = \nabla \cdot (\theta_0 \boldsymbol{u}_0)$	$\mathrm{div}(Y)$ vs $\varepsilon \nabla \cdot (\theta \boldsymbol{u}_X)$: cosine $+0.994$ for $\varepsilon < 0.6$, $+0.965$ overall, amplitudes 2.26×10^{-2} vs 2.10×10^{-2}
the expansion converges fast	per configuration, ε^1 alone carries 86% of the correction energy, $\varepsilon^1 + \varepsilon^2$ carries 99.0%, ε^3 carries 99.9%

4 Data sets and protocols

Table 3: Data sets used in this study. The public sets are from Menglik et al. [5].

set	n	what varies	purpose
random-BC	600	$Ra \sim \log U[10^2, 10^6]$, $\varepsilon \sim U[0.2, 1.9]$, 134 wall-mode families	calibrate the law, train surrogates
$\varepsilon \to 0$ checks	27	$\varepsilon = 0.02/0.05/0.10 \times Ra = 10^3/10^4/10^5$	test $R(0) = 0$
paired	20×5	only ε; Ra and the θ boundary stream fixed within a configuration	exact ε-blind floor, shape invariance
aspect 2:1	104	same law, geometry 2×1	external validity
tight rerun	10	the 10 worst-converged samples, tolerance $10^{-6} \to 10^{-8}$	noise control
public, random walls [5]	$4000 + 3 \times 200$	$\varepsilon \equiv 1.2$; the three held-out splits differ in wall dials and in whether the side walls are constant	external validation, cross-data-set transfer
public, classical cavity [5]	$5000 + 200$	constant hot/cold side walls, adiabatic top and bottom, $\varepsilon \equiv 1.2$	external validation of the prefactor
water (this work)	104	$Pr = 4.38$; BC/Ra/ε draws bit-identical to 104 of the random-BC samples	test of the fluid axis

Surrogates use residual learning $\hat{Y} = X + \mathcal{N}(\cdot)$, a U-Net of width 32 (or an FNO2d with 12 modes, width 32, four layers, 1.19 M parameters), Adam with cosine annealing, 200 epochs, batch 16, and targets normalised per channel. Two splits are used throughout: **extrapolation** (train $\varepsilon \leq 1.3$, test $\varepsilon > 1.3$, $n = 218$) and **interpolation** (stratified 2:1 in ε, $n = 200$). Errors are the per-channel relative L_2 error against Y; "floor" means $\hat{Y} = X$.

5 Results

5.1 The ε-law, calibrated over two decades

The exponent is window-averaged, and the curvature is real. The fit above uses the full two decades, including the 27 check points at $\varepsilon \leq 0.1$ where R is closest to its linear limit. Fitting only $\varepsilon \geq 0.8$ gives $p = 1.23 \pm 0.08/1.25 \pm 0.08/1.21 \pm 0.34/1.32 \pm 0.06$, and fitting each of the 20 paired configurations separately over $\varepsilon \in [0.4, 1.9]$ gives a median $p = 1.31/1.17/1.21/1.45$ (median $R^2 = 0.98$, `code/audit_v2.py`). That is the signature of $R = A\varepsilon(1 + \kappa\varepsilon)$ rather than of an exact power: a power law fitted to a curved function over a window that excludes small ε

Table 4: Power-law fit $R = A\varepsilon^p$ over the full two-decade window (627 samples). The intercept is fitted separately on the small-ε subset ($\varepsilon \leq 0.5$).

channel	$R = A\varepsilon^p$	A	R^2	intercept ($\varepsilon \leq 0.5$)
u_x	$p = 1.023 \pm 0.018$	0.147	0.84	$+0.0003 \pm 0.0047$
u_y	$p = 0.995 \pm 0.019$	0.193	0.82	$+0.0030 \pm 0.0062$
θ	$p = 1.024 \pm 0.077$	0.010	0.22	-0.0003 ± 0.0012
p	$p = 1.009 \pm 0.015$	0.148	0.87	$+0.0016 \pm 0.0030$

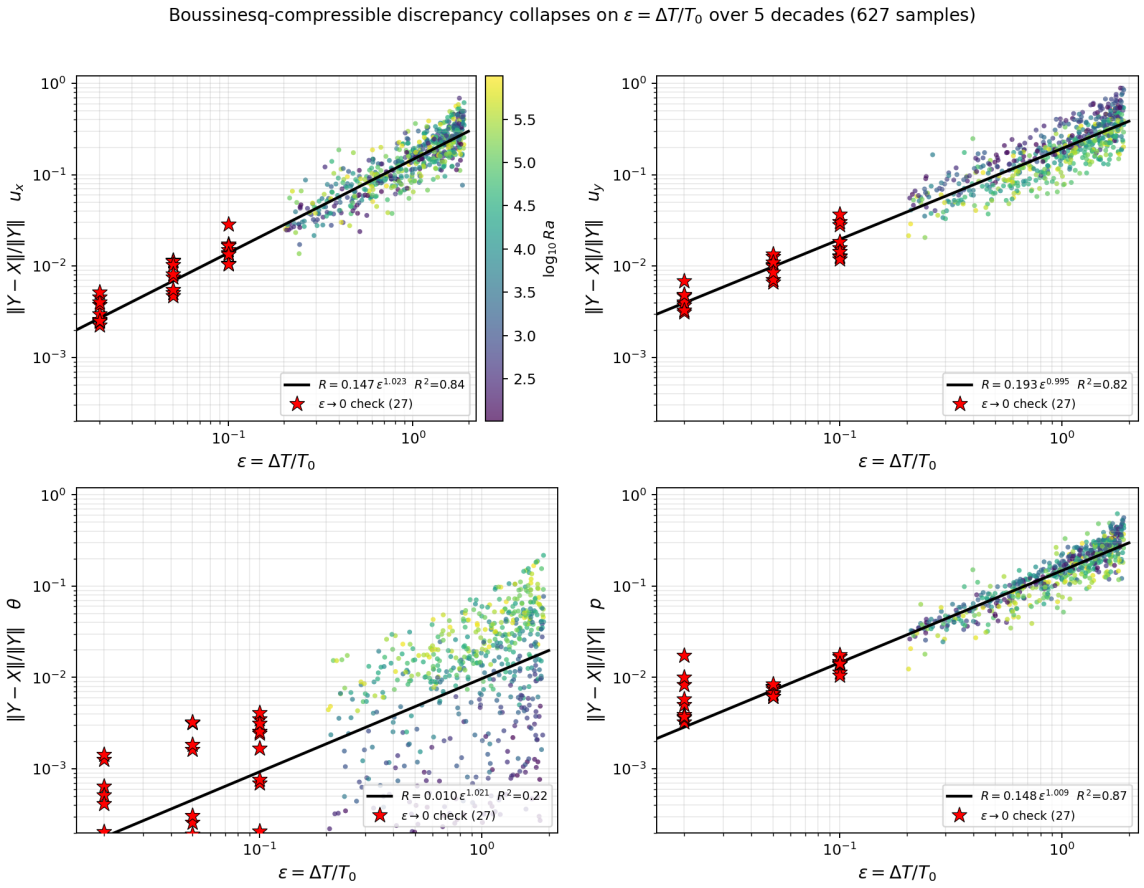


Figure 1: The Boussinesq–compressible discrepancy collapses on $\varepsilon = \Delta T/T_0$ over two decades. Points: 600 random-boundary-condition samples, coloured by $\log_{10} Ra$; stars: the 27 independent small-ε checks. Lines: the power-law fits of Table 4.

returns $p \approx 1 + \kappa\langle\varepsilon\rangle$. The quadratic fit $R = a\varepsilon + c\varepsilon^2 + b$ gives $c > 0$ in all four channels (u_x: $+0.0513 \pm 0.0133$, 3.9σ; u_y: $+0.0695$; θ: $+0.0060$; p: $+0.0539$) with $\kappa = c/a = 0.70/0.83/0.42/0.73$. We therefore state the law as **first order in** ε, $R = A\varepsilon(1+O(\varepsilon))$, quote $p = 1.02$ as the number a wide-window fit returns, and note that the ε^2 curvature is independently recovered by the network (Sec. 5.5, $\beta = 0.23$–0.42) and by the mode expansion (Sec. 3, $\varepsilon^1 + \varepsilon^2$ carry 99.0% of the correction energy).

Table 5: Two-parameter refinement, $R = A\varepsilon^p Ra^q$. The Ra trend for u_y and θ reproduces in the 2:1 geometry.

channel	unit cavity ($n = 600$) p / q / R^2	aspect 2:1 ($n = 104$) p / q / R^2
u_x	1.081 / $+0.003 \pm 0.006$ / 0.74	1.071 / -0.143 ± 0.019 / 0.71
u_y	1.016 / -0.083 ± 0.005 / 0.77	1.146 / -0.077 ± 0.019 / 0.69
θ	1.120 / $+0.514 \pm 0.015$ / 0.72	0.989 / $+0.447 \pm 0.045$ / 0.55
p	1.127 / -0.029 ± 0.005 / 0.83	1.057 / -0.154 ± 0.017 / 0.74

- ε-scaling is geometry-independent; the prefactor A is not ($0.147 \to 0.175$ for u_x, $0.148 \to 0.222$ for p, $0.193 \to 0.175$ for u_y, $0.010 \to 0.008$ for θ).
- Boundary-condition families barely matter *inside this family*: 134 wall-mode families, 133 free dummies. Adding them to the ε-only fit raises the adjusted R^2 by 2.3 pp (p), 2.9 pp (u_y), 3.3 pp (u_x) and 8.5 pp (θ)—the ε-scaling is not an artefact of which wall modes were drawn (`code/audit_numbers.py`).
- The θ channel needs one extra, cheaply available parameter, the advection strength of the base flow $\chi = \|\boldsymbol{u}\|\|\nabla\theta\|/\|\theta\|$: $R_\theta = \varepsilon(-0.0037 + 0.226\chi)$, R^2 $0.185 \to \mathbf{0.778}$ (Spearman 0.902), i.e. $R_\theta \approx 0.23\varepsilon\chi$. For $u_x/u_y/p$ adding χ does nothing. A single-input form of the same statement, $R_\theta/\varepsilon = c\hat{u}^a$, is calibrated on two independent generators in Sec. 5.7 and carries the same information with $R^2 = 0.90$.
- A pressure-channel preprocessing bug had to be fixed first (Sec. 5.2).

5.2 A pressure-channel artefact, and why it mattered

Pairing the two members' *raw* pressures mixes units: the compressible raw pressure carries a hydrostatic ramp 12× larger than its dynamic part, and the two weak forms use opposite pressure sign conventions. Before the fix the per-sample median of $|X_p|/|Y_p|$ was 0.065 and that of $\cos(X_p, Y_p)$ was -0.105, and the p-channel "law" was a constant ($R_p \approx 1.0$). Applying the recipe used by the public data set—subtract the mean, remove a least-squares linear-in-y fit, flip the sign of the Boussinesq member—gives 0.960 and cos = $+0.991$ under the same median convention, against 0.929 and $+0.986$ for the public training set, i.e. we now match their pairing to within 3% (`code/audit_v2.py`). The channel then obeys the same law as the velocities ($A = 0.148$, $p = 1.009$). All p-channel numbers reported here are post-fix.

5.3 The ε-blind ceiling, measured exactly

Within a configuration X is identical for all ε, so an ε-blind model must emit one field F for every ε it is applied at. Two reference protocols are exact rather than fitted: (A) train at the one ε a published data set offers (here $\varepsilon = 1.2$) and apply it everywhere, error $\|Y(\varepsilon) - Y(1.2)\|/\|Y(\varepsilon)\|$; (B) the single field that best serves the whole range, the ε-mean of Y. (B) is a least-squares compromise over the ε-range, **not a per-ε lower bound**—a field tuned to $\varepsilon = 0.8$ gives 0.067 at $\varepsilon = 0.4$, better than (B)'s 0.170 there, and below $\varepsilon \approx 0.8$ doing nothing beats both. Median over 20 configurations, channels $u_x/u_y/\theta/p$:

Table 6: The exact ε-blind floor. Protocol (A) is what a single-ε public data set forces; protocol (B) is the best single field over the whole range. Entries are medians over 20 configurations, ordered $u_x/u_y/\theta/p$.

ε	do nothing ($\hat{Y} = X$)	(A) train at $\varepsilon = 1.2$ only	(B) ε-mean field
0.4	0.060/0.098/0.009/0.061	0.143/0.219/0.021/0.146	0.170/0.244/0.023/0.177
0.8	0.127/0.203/0.019/0.129	0.078/0.118/0.011/0.082	0.108/0.143/0.013/0.126
1.2	0.211/0.328/0.030/0.220	0/0/0/0	0.035/0.036/0.003/0.051
1.6	0.322/0.481/0.044/0.347	0.112/0.151/0.014/0.131	0.079/0.123/0.012/0.085
1.9	0.472/0.616/0.058/0.547	0.251/0.304/0.031/0.346	0.225/0.286/0.028/0.281

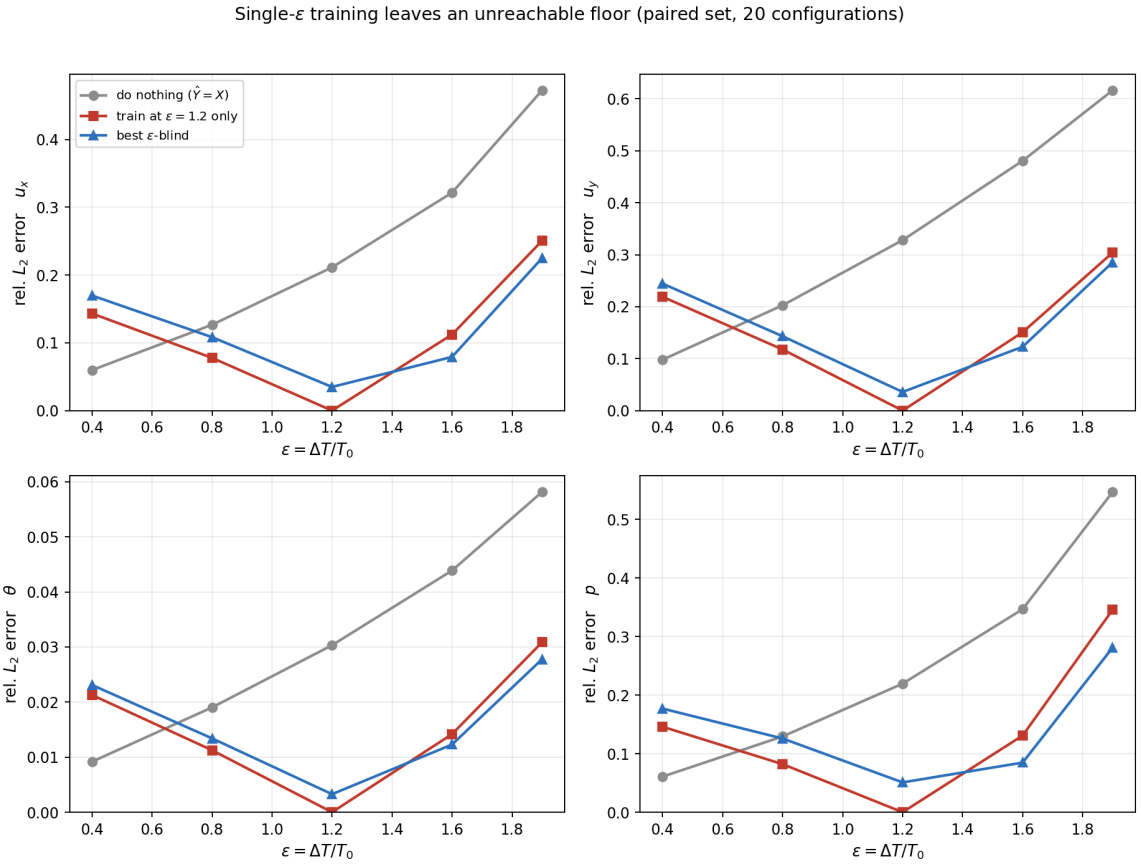


Figure 2: Single-ε training leaves an unreachable floor. Curves show "do nothing", training at $\varepsilon = 1.2$ only, and the best ε-blind field, for the 20 paired configurations.

Protocol (A) is what a single-ε public data set forces: 25% (u_x), 30% (u_y) and 35% (p) error at $\varepsilon = 1.9$, 8% (u_x) already at $\varepsilon = 0.8$, and—the point for practice—at $\varepsilon = 0.4$–0.8 "do nothing" is *better* than it. Protocol (B), the best an ε-blind model could possibly do over the range, still leaves 22.5/28.6/28.1% at $\varepsilon = 1.9$. The whole effect is one line of arithmetic, and the paired set measures it exactly (`code/exp_paired_final.py`; the diagonal is identically zero because the same ε is the same solution):

Table 7: Exact transfer error $R(u_x)$ when a field computed at one ε (columns) is applied at another (rows).

$R(u_x)$ applied at $\downarrow$ / taken from $\rightarrow$	0.4	0.8	1.2	1.6	1.9
$\varepsilon = 0.4$	0	0.067	0.143	0.235	0.353
$\varepsilon = 0.8$	0.068	0	0.078	0.183	0.305
$\varepsilon = 1.2$	0.149	0.081	0	0.110	0.243
$\varepsilon = 1.6$	0.265	0.199	0.112	0	0.140
$\varepsilon = 1.9$	0.402	0.328	0.251	0.142	0

The error is set by the distance in ε and by nothing else—which is precisely the quantity $\varepsilon P(X)$ puts back into the model.

5.4 Network baselines: the correction is information, not capacity

Table 8 reports a U-Net of width 32, 200 epochs, mean of three seeds, identical protocol for all columns (largest standard deviation over every cell: 0.0051).

Table 8: U-Net ablation. A sees only the four Boussinesq fields; B also receives ε as a constant input channel. "Energy recovered by B" is $1 - (R_B/R_{\text{do nothing}})^2$.

split	channel	A (no ε)	B (ε input)	gain	do nothing	energy recovered by B
extrapolation ($n = 218$)	u_x	0.2289	**0.1508**	1.52×	0.3018	75%
	u_y	0.2733	**0.1725**	1.58×	0.3731	79%
	θ	0.0406	**0.0255**	1.59×	0.0556	79%
	p	0.2689	**0.1791**	1.50×	0.3320	71%
interpolation ($n = 200$)	u_x	0.1530	**0.0673**	2.27×	0.2203	91%
	u_y	0.1796	**0.0714**	2.52×	0.2683	93%
	θ	0.0261	**0.0097**	2.69×	0.0381	94%
	p	0.1792	**0.0667**	2.69×	0.2386	92%

Model A's error (0.2289) and the best possible ε-blind model on the paired set at $\varepsilon = 1.9$ (0.2249, Sec. 5.3) agree to within 2% while being measured on two different data sets: the ceiling is informational, not architectural. Shallow learners plateau in the same place. A linear ridge regression and an RBF kernel ridge on the raw four-field input—the same information the network gets—reach u_x 0.2282 and 0.2222 in extrapolation, and adding ε as an extra column barely moves them (0.2188 and 0.2171, against 0.0937 for the structured model), because neither can represent a product of an ε-scale with a spatial field. A classical POD-ROM (44–52 modes, 99.9% of the snapshot energy) whose coefficients are regressed on $(\log_{10} Ra, \varepsilon)$ lands at 0.2937 with ε and 0.3001 without, i.e. at the level of doing nothing, and a purely linear surrogate that also sees the POD coefficients of X reaches only 0.2318. The plateau is not capacity—it is that the quantity to be added is absent from the input (`code/exp_shallow.py`, `code/exp_pod.py`).

5.5 The physics prior: $D = \varepsilon P(X)$

Same backbone, only the output structure changes (ε still read from a constant channel, target still D):

Table 9: Output-factorisation ablation. P1 multiplies ε outside the network; P2 adds a learned quadratic term. β is learned per channel and is the same sign and order as the ε^2 coefficient of the expansion.

split	variant	u_x	u_y	θ	p	learned β
extrapolation	plain U-Net	0.1508	0.1725	0.0255	0.1791	—
	$D = \varepsilon P(X)$	0.0937	0.1060	0.0141	0.1126	fixed 0
	$D = (\varepsilon + \beta\varepsilon^2)P(X)$	**0.0867**	**0.0951**	**0.0131**	**0.1007**	0.418/0.321/0.343/0.338
interpolation	plain U-Net	0.0673	0.0714	0.0097	0.0667	—
	$D = \varepsilon P(X)$	0.0545	0.0524	0.0078	0.0454	fixed 0
	$D = (\varepsilon + \beta\varepsilon^2)P(X)$	**0.0507**	**0.0488**	**0.0073**	**0.0410**	0.286/0.232/0.252/0.265

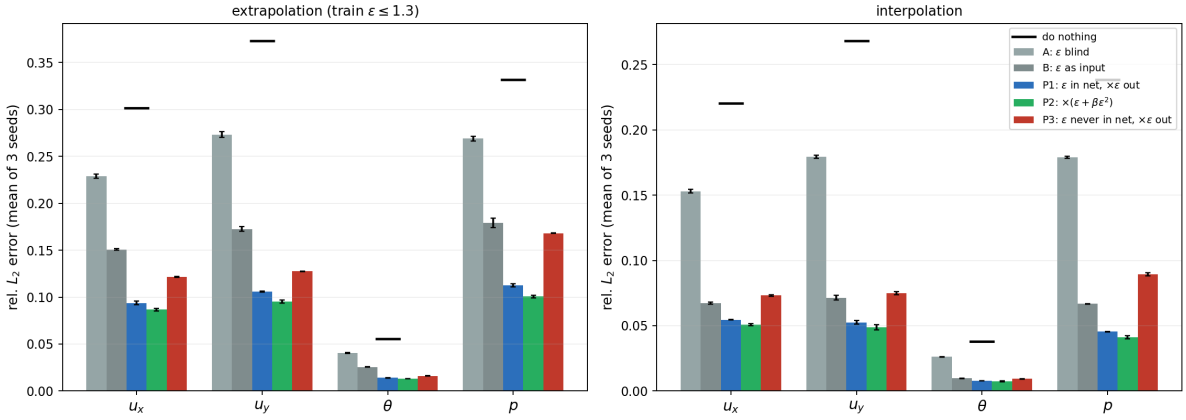


Figure 3: The prior ablation for extrapolation (left) and interpolation (right). A: ε-blind. B: ε as input. P1: ε in the network, multiplied outside. P2: learned $(\varepsilon + \beta\varepsilon^2)$ factor. P3: ε never enters the network.

Two things happen at once. In extrapolation the factorised output beats the plain one by **1.61**× (u_x) and 1.59–1.81× in the other channels; the quadratic form adds a further 7.6–10.6% on top. And the network *discovers* a positive curvature on its own, $\beta = 0.23$–0.42, the same sign and order as the ε^2 term of the expansion in Sec. 3—an independent check of the asymptotic expansion.

A Fourier neural operator (1.19 M parameters) shows the same pattern against its ε-blind version: u_x 0.2287 → **0.1334** with the prior and ε multiplied back outside the network (1.71×), statistically indistinguishable from an explicit ε input channel (0.1334 ± 0.0004 against 0.1342 ± 0.0008). On u_y and θ the prior is in fact better (0.1387 against 0.1456; 0.0177 against 0.0191); only on p does the ε channel win (0.1536 against 0.1734). The prior's value is therefore not that it beats an ε-aware model—it is that it matches one while needing no ε in the input (Secs. 5.6, 5.8).

5.6 Data efficiency

Table 10: Learning curve on the fixed interpolation test set (200 samples), mean of three seeds. Entries are $u_x/u_y/\theta/p$. P3 keeps ε outside the network entirely.

n	B: ε as input	P3: predict $P(X)$, ε outside
0 (do nothing)	0.2203/0.2683/0.0381/0.2386	same
50	0.1831/0.1946/0.0269/0.1838	**0.1308/0.1364/0.0195/0.1501**
100	0.1280/0.1422/0.0199/0.1348	**0.0990/0.1022/0.0136/0.1150**
200	0.0974/0.1070/0.0147/0.0998	**0.0827/0.0866/0.0114**/0.1005
400	**0.0674/0.0714**/0.0097/**0.0672**	0.0735/0.0743/0.0093/0.0896

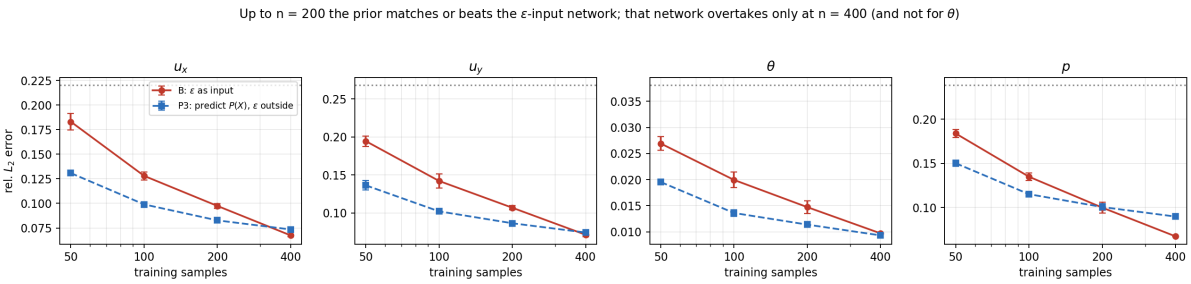


Figure 4: Learning curves. Up to $n = 200$ the prior matches or beats the ε-input network; the latter overtakes only at $n = 400$, and not for θ.

The naive expectation—that a structured model trades in-distribution accuracy for extrapolation—is wrong at small n. At $n = 50$ the prior is 1.4× better than the ε-input network (u_x 0.1308 against 0.1831, a 9σ separation across three seeds), and it is still ahead at $n = 200$ in u_x (7.3σ), u_y (7.6σ) and θ (3.6σ), with p already level (0.13σ) there. The crossover sits between $n = 200$ and 400 in u_x and u_y (log-linear interpolation: $n \approx 315$ and 360, i.e. near 300 in u_x), essentially at $n = 200$ in p, and θ never crosses—at $n = 400$ it is still 2.5σ behind. Concretely, the prior at $n = 200$ reaches what the ε-input network needs ≈ 270 samples (u_x) and 290 (u_y) to reach. So the prior is two things at once: the cheaper model when data is scarce, and—the point that matters for reuse—a model that needs no ε in its input, which is what makes a single-ε data set reusable.

5.7 External validation on an independent, published data set

`NurshatMenglik/NeuralConvection_2D_TrainTest` (v2; 4000 training pairs plus 200 in-distribution test pairs, CC BY 4.0), generated by another group with their own pipeline, at a **single** $\varepsilon = 1.2$, $Ra \in [10^2, 3 \cdot 10^6]$ [5].

Conventions match. Recomputing their published `relL2_*` columns as $\|Y-X\|/\|X\|$ from the tensors reproduces them exactly (0.14233/0.13749/0.01356/0.19070 for sample 0); their pressure amplitude ratio is 0.929 against 0.931 for our fixed pairing.

The law reproduces. On the $\varepsilon = 1.2$ slice:

Table 11: External validation at $\varepsilon = 1.2$: public data against our own slice. Medians with p10–p90 in parentheses.

channel	public 4000 (median, p10–p90)	our data at $\varepsilon \in [1.1, 1.3]$ ($n = 64$)
u_x	0.1797 (0.112–0.300)	0.1839 (0.126–0.284)
u_y	0.2435 (0.141–0.403)	0.2250 (0.150–0.395)
θ	0.0228 (0.001–0.071)	0.0232 (0.001–0.056)
p	0.1858 (0.117–0.246)	0.1889 (0.128–0.262)

Even the secondary trends reproduce. The correlation of R with $\log_{10} Ra$ is $+0.133/-0.419/+0.647/-0.198$ on their $\varepsilon \equiv 1.2$ data and $+0.032/-0.511/+0.662/-0.263$ on our $\varepsilon \in [1.1, 1.3]$ slice (64 samples, $u_x/u_y/\theta/p$). The θ channel—the one whose scatter we explain next—has $r = +0.66$ on ours and $+0.65$ on theirs, from independently generated data.

Five splits, four prefactors, one predictable pattern. The release publishes three further held-out splits generated by the same generator and the same solver commit (`v6_freefall_`

20260921, raw_both_deramp pressure convention; the OOD split carries the later tag v6_freefall_ood_20260925): test_in_distribution (200 random-wall pairs), test_ood (200 random-wall pairs whose vertical-wall Fourier dials lie outside the training range) and test_legacy (200 pairs of the classical cavity: constant hot and cold side walls, adiabatic top and bottom). We had already downloaded train_legacy, 5000 classical-cavity pairs. Recomputing all of relL2_u_x, relL2_u_y, relL2_T, relL2_p for **every sample of all five splits (38 400 values: 9600 samples × 4 channels)** under our conventions reproduces the published numbers to within 1.2×10^{-7} (float32 round-off; code/verify_conventions.py), so nothing below is a convention artefact. At the fixed $\varepsilon = 1.2$ the medians of $R = \|Y - X\|/\|Y\|$ are:

Table 12: Median relative correction on all five public splits, at $\varepsilon \equiv 1.2$.

split	n	u_x	u_y	θ	p
train (random walls)	4000	0.1797	0.2435	0.0228	0.1858
test_in_distribution	200	0.1822	0.2388	0.0220	0.1863
test_ood (harder wall dials)	200	0.1567	0.2157	0.0214	0.1671
train_legacy (classical cavity)	5000	0.2988	0.3362	0.0926	0.1989
test_legacy (classical cavity)	200	0.2984	0.3390	0.0836	0.1983

The exponent cannot be re-tested here—every split sits at one ε—but the prefactor is put under pressure and it moves: by 1.66× (u_x), 1.38× (u_y), 4.05× (θ) and 1.07× (p) between the classical cavity and the random-wall families at the same ε, and train_legacy and test_legacy agree to three digits in u_x and p (θ: 0.0926 vs 0.0836, 10%), which is the internal consistency check for the whole comparison. That is a statement about the flow, not a defect of the law, and the θ channel makes the mechanism explicit. Writing $\hat{u} = \max \|\boldsymbol{u}_X\|$, the peak speed of the Boussinesq field (hence available to any surrogate):

Table 13: The θ-channel prefactor is the flow strength: $R_\theta/\varepsilon = c\hat{u}^a$ on six data sets from two independent generators.

data set	n	$R_\theta/\varepsilon = c\hat{u}^a$	R^2
ours, random walls, $Pr = 0.71$	600	$a = 1.864 \pm 0.026$, $c = 0.544$	0.90
public train, random walls	4000	$a = 1.902 \pm 0.010$, $c = 0.602$	0.91
public test_in_distribution	200	$a = 1.977 \pm 0.041$, $c = 0.691$	0.92
public test_ood	200	$a = 1.931 \pm 0.041$, $c = 0.561$	0.92
public train_legacy, classical cavity	5000	$a = 2.360 \pm 0.002$, $c = 2.063$	**0.995**
public test_legacy, classical cavity	200	$a = 2.368 \pm 0.011$, $c = 2.095$	**0.996**

Two independent groups, two independent generators, six data sets: the random-wall families all give $c = 0.54$–0.69, the classical cavity—whose flow is a single strong circulation with $\hat{u}$ 41% above the public random-wall median and 56% above ours—gives $c = 2.06$–2.10 (code/exp_uhat_law.py). The same reading explains the residual scatter in $u_x/u_y/p$ in Sec. 5.1: A is a functional of the flow, and the flow is in the input.

5.8 Cross-data-set transfer: the payoff, and its boundary

Two public single-ε data sets are available: 4000 random-wall pairs (our boundary-condition family) and 5000 classical-cavity pairs (a different one). Both sit at $\varepsilon \equiv 1.2$. We train the same U-Net on each—once with the plain output (target D), once with the prior (target D/ε, and ε multiplied back outside the network)—and apply the result to **our own 600 samples with no retraining**, at ε from 0.2 to 1.9, values the training data never contained.

Three readings, in order of how much they matter for practice.

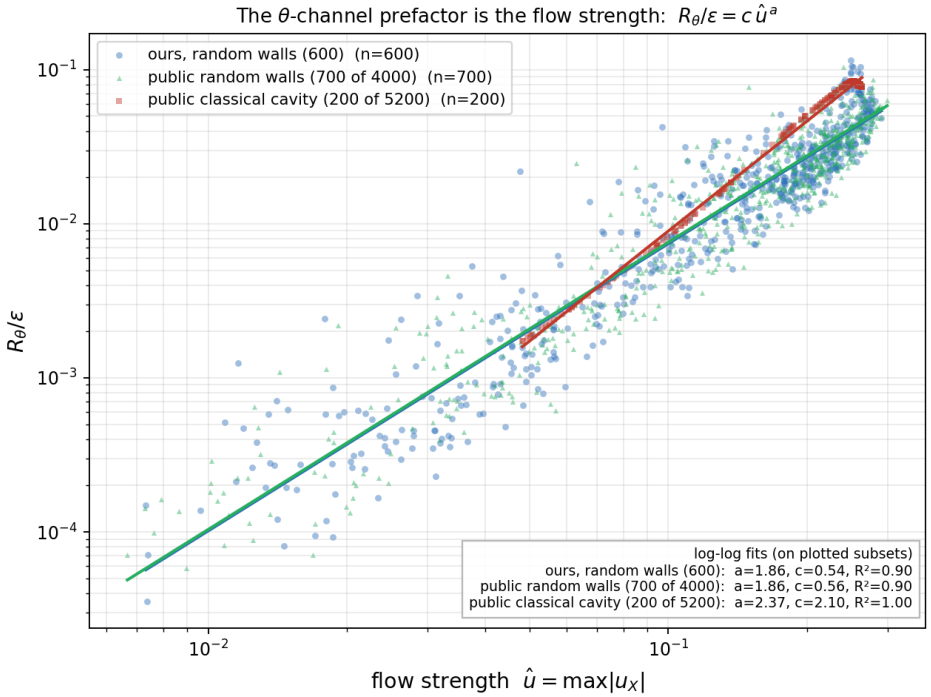


Figure 5: The θ-channel prefactor collapses on the peak Boussinesq speed $\hat{u}$ across two independent generators and two boundary-condition families.

Table 14: Transfer from a single-ε public data set to our own $\varepsilon \in [0.2, 1.9]$ samples. Entries are $u_x/u_y/\theta/p$.

trained on	model	$\varepsilon > 1.3$, $n = 218$	full ε, $n = 200$
do nothing	$\hat{Y} = X$	0.3018/0.3731/0.0556/0.3320	0.2203/0.2683/0.0381/0.2386
public 4000, random walls, $\varepsilon \equiv 1.2$	plain	0.1443/0.1730/0.0226/0.1838	0.1199/0.1441/0.0189/0.1435
public 4000, random walls, $\varepsilon \equiv 1.2$	**prior**	**0.0932/0.1085/0.0119/0.1322**	**0.0657/0.0769/0.0077/0.0891**
our own 382 training samples	prior	0.1250/0.1321/0.0170/0.1719	0.0791/0.0811/0.0103/0.0947
public 5000, classical cavity, $\varepsilon \equiv 1.2$	plain	0.2919/0.3711/0.0490/0.3192	0.2444/0.3612/0.0484/0.3217
public 5000, classical cavity, $\varepsilon \equiv 1.2$	prior	0.3092/0.4730/0.0561/0.3280	0.2281/0.3498/0.0422/0.2584

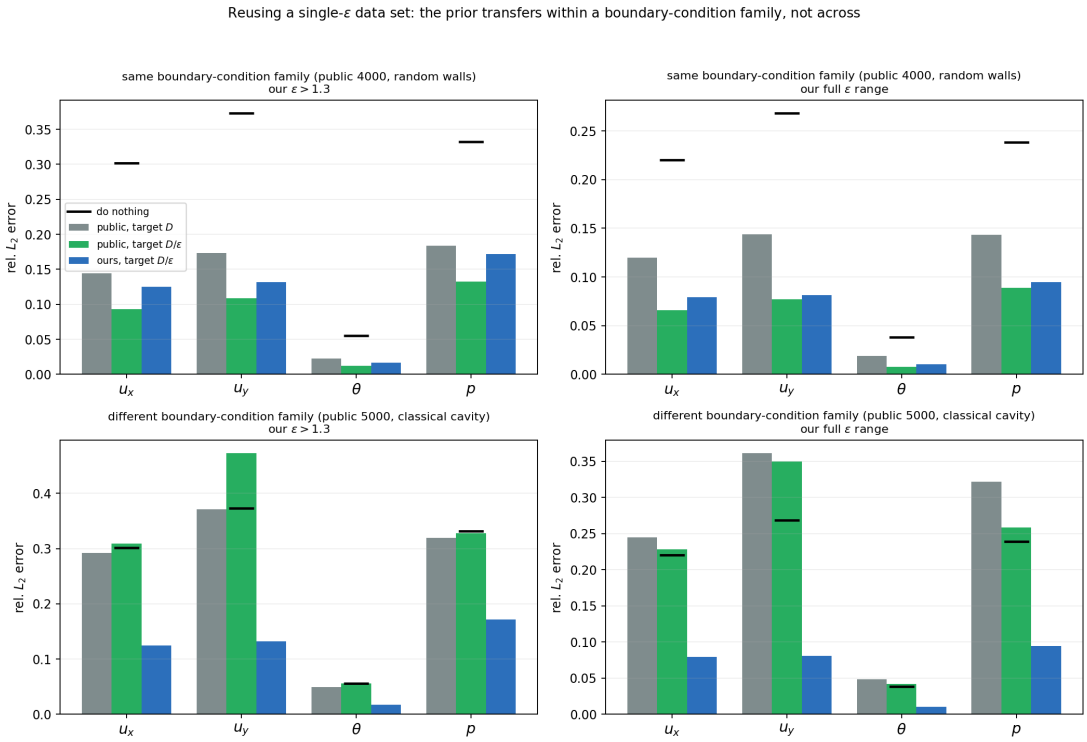


Figure 6: Reusing a single-ε data set. Within the boundary-condition family the prior transfers and beats the plain model; across families it fails, and can be worse than doing nothing.

- **Within the family, the prior is what makes a single-ε data set reusable.** The identical architecture, budget and data give 0.1443 (u_x) with the plain output and 0.0932 with the prior—a factor 1.55—and 3.2× better than doing nothing, on ε the data never contained. It is even better than the same prior trained on our own 382 samples (0.1250), because it sees ten times more configurations. This is the concrete content of "the ε dependence need not be in the data".

- **The prior also fixes the failure mode of the plain model.** The plain model's error at unseen ε is largest where the training ε is furthest away (its correction is a fixed field, Sec. 5.3), whereas the prior's error stays proportional to how wrong P is, uniformly in ε.

- **Across boundary-condition families the same trick fails, and the prior makes it worse.** Trained on the classical cavity, the plain model lands at 0.2919 against a floor of 0.3018—it learned almost nothing transferable—and the prior model is *worse* than the plain one (0.3092 at $\varepsilon > 1.3$): it confidently multiplies a P learnt on the wrong flow by an ε that grows to 1.9. This is exactly what Sec. 5.7 predicts, and it is the same number: A is a functional of the flow, the classical cavity's flow has $\hat{u}$ 41% above the random-wall median, and a surrogate trained on that family predicts the wrong A. Reusing published data therefore requires the boundary-condition family to be represented; the law tells you in advance which reuse will work (`code/exp_public_transfer.py`, `code/exp_public_legacy_transfer.py`).

5.9 A second geometry (aspect ratio 2:1, 104 samples)

Same generator, same Ra and ε ranges, cavity twice as wide. The ε exponent is unchanged ($p = 1.09/1.16/0.93/1.08$ for $u_x/u_y/\theta/p$, 0.7σ, 1.8σ, 0.3σ and 0.7σ away from the unit-square values), while the prefactor moves ($A = 0.175/0.175/0.008/0.222$ versus $0.147/0.193/0.010/0.148$). The Rayleigh trend is stronger for u_x and p here ($q = -0.14, -0.15$ vs $0.00, -0.03$) and reproduces

for u_y and θ. **The ε-scaling is geometry-independent; calibration of A is not**—which is exactly why the surrogate takes X as input rather than a table of A. Changing the fluid instead (Pr 0.71 $\rightarrow$ 4.38) moves neither the exponent nor the prefactor (Sec. 5.12).

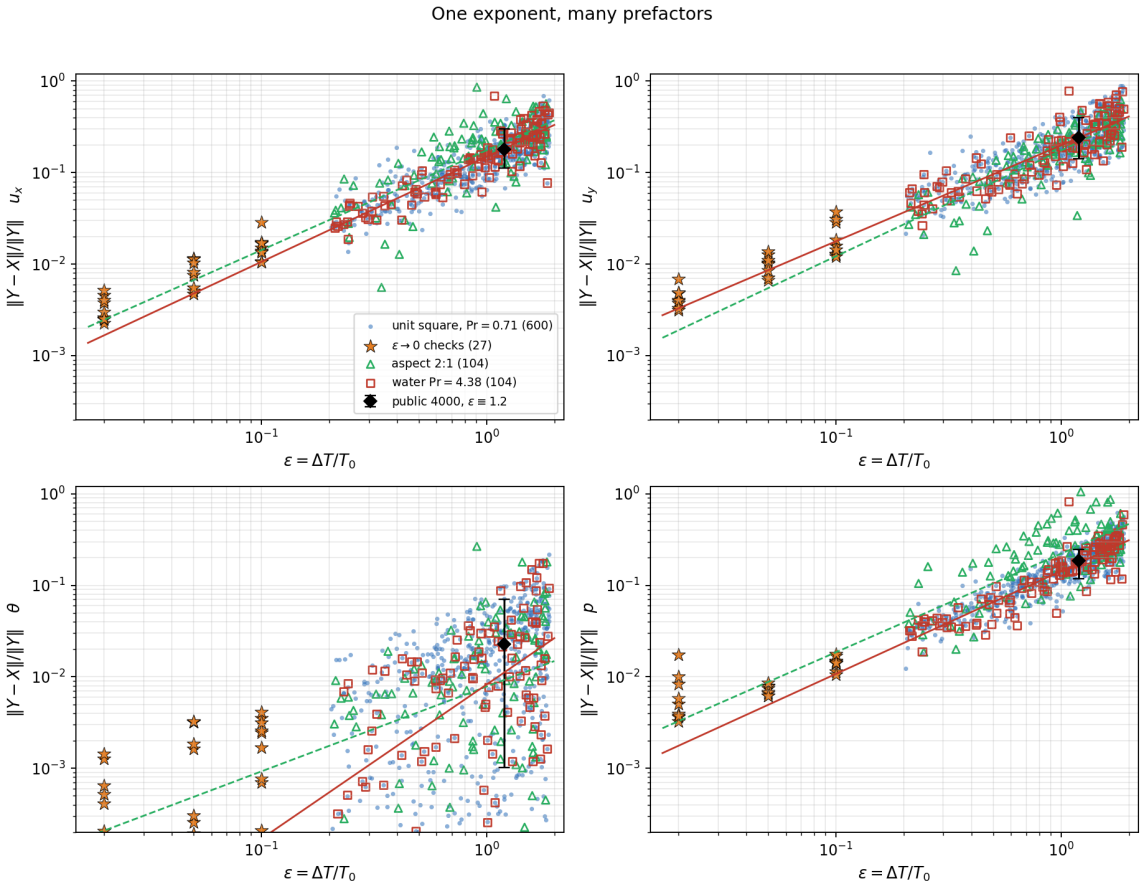


Figure 7: One exponent, many prefactors: unit square ($Pr = 0.71$), aspect 2:1, water ($Pr = 4.38$), and the public data set at $\varepsilon = 1.2$.

5.10 Solver noise is not the source of the law

- Convergence quality over 600 samples (compressible member): median relative change 3.75×10^{-6}, p90 1.09×10^{-4}, worst 7.04×10^{-4}.
- Dropping the 65 samples with relative change $> 10^{-4}$ changes the u_x fit from $0.1809\varepsilon - 0.0212$ to $0.1813\varepsilon - 0.0221$—unchanged.
- Re-running the 10 worst samples with `steady_tol` tightened from 10^{-6} to 10^{-8} leaves the solution **bit-identical** ($|\Delta Y|/|Y| = 0$). The reason is structural: the compressible march terminates on the zero-iteration guard, not on `steady_tol`, so the residual reported is the fixed-point residual of the discrete solution, not unfinished iteration.

5.11 Cost and speed-up

The resulting factors are $\mathbf{5.4 \times 10^3}$ per sample and $\mathbf{6.9 \times 10^4}$ in batches.

Table 15: Wall-clock cost on one RTX 4060 Laptop, 64^2 mesh, eight-way parallel solver.

	median	mean	p90	max
paired solve, 600 samples	33.9 s	38.0 s	59.9 s	510 s
paired solve, paired set	55.8 s	67.7 s	136 s	202 s
surrogate, GPU, batch 1	6.3 ms			
surrogate, GPU, batch 256	0.49 ms/sample			
surrogate, CPU, 1 thread	14.0 ms			

5.12 A third fluid: $Pr = 4.38$ (water)

Pr and ε do not mix. Pr enters the coefficients of both members ($Re^{-1} = (Pr/Ra)^{1/2}$) and hence the Boussinesq solution X, but the ε-expansion of Sec. 3 never multiplies the two. The prediction is therefore sharp and asymmetric: the exponent must not move, while the prefactor may, because X itself has changed.

We generated 104 further pairs at $Pr = 4.38$ (water at 20°C) that are **bit-identical to the first 104 samples of the main batch in every other respect**: same seed stream, hence the same wall-temperature Fourier draws, the same Ra and the same ε. The two families are therefore a strict paired experiment in which the fluid is the only difference (the Mach-number law is unchanged, and Ma is inert, Sec. 2.2).

Table 16: The fluid axis: 104 strictly paired air/water configurations. The exponent is unchanged; the prefactor moves by less than 5% on the paired comparison.

	ε	Ra	u_x	u_y	θ	p
$Pr = 0.71$, air (unit square)	0.2–1.9	10^2–10^6	0.1468	0.1934	0.0097	0.1482
$Pr = 4.38$, water (104 paired)	0.21–1.89	$1.1 \cdot 10^2$–$8.9 \cdot 10^5$	0.1496	0.1985	0.0082	0.1426
$Pr = 4.38$, p			1.150 ± 0.054	1.049 ± 0.063	1.686 ± 0.238	1.124 ± 0.054

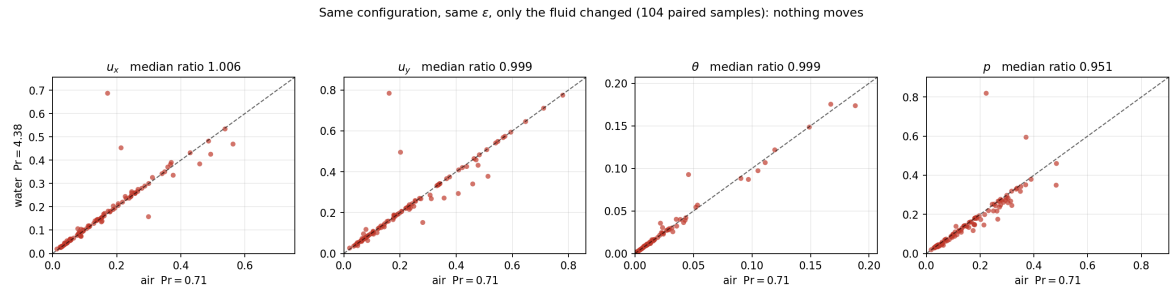


Figure 8: Same configuration, same ε, only the fluid changed: air ($Pr = 0.71$) against water ($Pr = 4.38$), 104 paired samples.

On the 104 matched configurations the paired ratios $R_{\text{water}}/R_{\text{air}}$ have medians **1.006**, **0.999**, **0.999** and **0.951** ($u_x/u_y/\theta/p$; p10–p90 0.99–1.12, 0.90–1.03, 0.95–1.06, 0.82–1.00), and the paired exponents agree to three digits: 1.150 vs 1.157, 1.049 vs 1.047, 1.686 vs 1.668, 1.124 vs 1.109. A 6.2-fold change in Prandtl number—air to water—moves the Boussinesq–compressible discrepancy by less than 1% on the same configuration.

Pr is therefore not a break-symmetry axis either, and the law's two ingredients separate cleanly: ε sets the exponent, the flow sets the prefactor, and the fluid does neither. Keeping the strict pairing matters for the last digit—the unpaired fits give θ 0.0097 → 0.0082 (a 15% move, the same direction as the 5% paired move) and u_x/u_y +2–3%—which is the same statement as Sec. 5.9 (a geometry moves A) and Sec. 5.7 (boundary-condition families move it by up to 4×).

6 Limitations

1. **Two dimensions only.** A three-dimensional pair at $res = 16$ costs ≥ 23 min for the Boussinesq member alone (1386 s for 21 steady steps) and the compressible member was still unfinished when a 30-minute wall-clock cap killed it (`code/run_3d_probe.sh`, `timeout 1800`); the historical $res = 24$ probe at $\varepsilon = 1.2$ never converged (Newton backtracking to $dt = 3.7 \times 10^{-4}$). A 3-D ε-sweep is out of reach on this machine and we do not claim one.

2. Ma **is inert here**, so the break-symmetry group is one-dimensional rather than two. At larger Ma this would have to be redone.

3. $\varepsilon < 2$, imposed by the equation of state (the cold wall must stay positive).

4. The two-parameter fits of Sec. 5.1 leave 26/23/28% of the variance unexplained in $u_x/u_y/\theta$ and 17% in p; the $\hat{u}$ form of Sec. 5.7 closes that gap for θ and leaves the same residual flow-dependence in the other three channels.

5. The prefactor A is configuration-dependent and must be learned: it moves with the geometry (Sec. 5.9) and by 1.66× (u_x) / 1.38× (u_y) / 4.05× (θ) / 1.07× (p) between boundary-condition families of an independent data set (Sec. 5.7), while it is insensitive to the fluid (Pr, Sec. 5.12).

6. The public data set and ours differ by 0.08% in Ra_{eff} at $\varepsilon = 1.2$ ($1.2/1.095^2 = 1.000813$); their v2 release corrects this and the two now agree.

7 Conclusion

We set out to build a surrogate for compressible natural convection and found that the object it has to reproduce—the gap between the Boussinesq solution and the compressible one—is governed by a single parameter in an exact and predictable way.

1. **One parameter.** In this non-dimensionalisation $Fr^2 = \Delta T/T_0$ identically, so the break-symmetry axis is $\varepsilon = \Delta T/T_0$ and nothing else; Mach-number effects are $O(Ma^2)$ and are invisible at the values reached here.

2. **A clean law.** $R = \|Y - X\|/\|Y\| = A\varepsilon^p$ with $p = 1.023 \pm 0.018$, 0.995 ± 0.019, 1.024 ± 0.077, 1.009 ± 0.015 (u_x, u_y, θ, p), an intercept indistinguishable from zero, the exponent quoted over the full two-decade window (from $\varepsilon = 0.8$ upwards the same points give 1.23–1.32, the ε^2 curvature of Sec. 5.1), and a 134-family boundary-condition description (133 dummies) that adds only 2–3 pp of adjusted R^2 to the velocity and pressure channels. The exponent is geometry-independent and fluid-independent; only the prefactor A must be calibrated, which is why the surrogate takes X as input rather than a table of A.

3. **A reason.** The ε-expansion of the compressible member reproduces the Boussinesq system at $O(1)$ and produces the correction through $\nabla \cdot \boldsymbol{u}_1 = \nabla \cdot (\theta_0 \boldsymbol{u}_0)$ and the density-modulated inertia. The linearised operator does not contain ε, so the correction factorises, $D = \varepsilon P(X) + \varepsilon^2 Q(X) + \ldots$, with a shape that does not rotate. All five consequences of the expansion are confirmed numerically.

4. **A design rule.** Predicting $P(X)$ and multiplying by ε outside the network—four lines of code—converts the law into a prior that (i) beats the plain architecture by 1.61× in extrapolation, (ii) lets the network discover the ε^2 term itself, (iii) works the same way for a Fourier neural operator, and (iv) makes a public data set that exists at a single temperature contrast reusable at others—0.093 (u_x) on $\varepsilon > 1.3$ against 0.144 for the plain model and

0.302 for doing nothing—which is the difference between a surrogate that must be retrained and one that can be applied. The scope of that reuse is set by Sec. 5.7: it holds within a boundary-condition family and fails across one, predictably.

The honest limits are in Sec. 6: two dimensions, $\varepsilon < 2$, one-dimensional break symmetry, and a prefactor whose two-parameter fits leave 26% (u_x), 23% (u_y) and 28% (θ) of the variance unexplained. None of them touch the claim that matters for practice, which is that the ε dependence is known in closed form and should not be left to the network to discover from data.

8 Reproducibility

All scripts are in the `code/` directory of the project (Chinese-path Windows workspace, executed in WSL2 with dolfinx 0.7.3 for the solver and PyTorch 2.10 for the surrogates); raw data, logs and JSON results are under `~/nc_run/`.

Table 17: Scripts and the results they produce.

script	what it produces
`exp_eps_law.py`	the 627-sample law (Sec. 5.1)
`repack_p_fixed.py`	the corrected pressure channel (Sec. 5.2)
`exp_paired_final.py`	the ε-blind floor and shape invariance (Sec. 5.3)
`exp_modes.py`	the ε^k mode expansion (Sec. 3)
`exp_chi.py`	the χ explanation for θ (Sec. 5.1)
`harness.py`, `exp_ml_final.py`	the unified surrogate ablation: A/B/P1/P2/P3, three seeds (Secs. 5.4–5.6)
`fno_lib.py`, `exp_fno_final.py`	the FNO baseline under the same protocol (Sec. 5.5)
`exp_lc_final.py`	learning curves (Sec. 5.6)
`exp_pod.py`	the POD-ROM / linear-surrogate baselines (Sec. 5.4)
`exp_p2_fixed.py`, `exp_prior_ablation.py`, `exp_fno.py`, `exp_learning_curve.py`	first-generation runs of the same experiments (superseded by the three scripts above; their JSON outputs were deleted as stale)
`exp_public_transfer.py`	external validation and transfer (Secs. 5.7–5.8)
`exp_aspect2.py`	the 2:1 geometry (Sec. 5.9)
`exp_tight.py`, `verify_noise.py`, `exp_final_numbers.py`	noise control and timing (Secs. 5.10–5.11)
`exp_water.py`	the strictly paired $Pr = 4.38$ fluid study (Sec. 5.12)
`verify_conventions.py`	the 38 400-value convention audit against the published `relL2` columns (Sec. 5.7)
`exp_public_splits.py`, `exp_legacy5000.py`, `exp_uhat_law.py`	the five public splits and the $\hat{u}$ law (Sec. 5.7)
`audit_v2.py`	every robustness number the review asked for: p vs fitting window, κ, pressure-pair conventions, $\hat{u}$ medians, the 2:1 σ distances
`make_figs.py`, `make_figs2.py`	every figure, from the JSON results
`make_tables.py`	regenerates every table from the JSON results

Code and every machine-readable result are mirrored at `https://github.com/sheep-cloud12138/Algorithms-instead-of-simulations` (working branch `dev`; `master` carries snapshots that have passed an independent review). The raw solver output (600 + 104 + 104 + 27 paired samples, ∼6 GB) stays on the machine described above; every number in this paper is regenerated from the JSON results in `~/nc_run/p7/` by reporting scripts, not transcribed by hand.

References

[1] Matteo Giacomini and Pedro Díez. Surrogates for physics-based and data-driven modelling of parametric systems: Review and new perspectives. *Archives of Computational Methods in Engineering*, 33(6):8519–8551, 2026. doi: 10.1007/s11831-026-10552-4.

[2] Chuizheng Meng, Sam Griesemer, Defu Cao, Sungyong Seo, and Yan Liu. When physics meets machine learning: A survey of physics-informed machine learning. *Machine Learning for Computational Science and Engineering*, 1(1):20, 2025. doi: 10.1007/s44379-025-00016-0.

[3] E. A. Spiegel and G. Veronis. On the Boussinesq approximation for a compressible fluid. *The Astrophysical Journal*, 131:442–447, 1960. doi: 10.1086/146849.

[4] Donald D. Gray and Aldo Giorgini. The validity of the Boussinesq approximation for liquids and gases. *International Journal of Heat and Mass Transfer*, 19(5):545–551, 1976. doi: 10.1016/0017-9310(76)90168-X.

[5] Nurshat Menglik, Alex Shao, and David Hyde. A neural surrogate approach for simulating natural convection problems, 2026. arXiv:2606.25259. doi:10.48550/arXiv.2606.25259.

[6] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021. arXiv:2010.08895.

[7] Quincy Huhn, Mauricio Tano, Jean C. Ragusa, and Youngsoo Choi. Parametric dynamic mode decomposition for reduced order modeling. *Journal of Computational Physics*, 475: 111852, 2022. doi: 10.1016/j.jcp.2022.111852.

[8] Feng Feng, Yubai Li, Zhihua Chen, Wei-Tao Wu, Jiang-Zhou Peng, and Mei Mei. Graph convolution network-based surrogate model for natural convection in annuli. *Case Studies in Thermal Engineering*, 57:104330, 2024. doi: 10.1016/j.csite.2024.104330.

[9] Jiang-Zhou Peng, Nadine Aubry, Yubai Li, Mei Mei, Zhihua Chen, and Wei-Tao Wu. Physics-informed graph convolutional neural network for modeling geometry-adaptive steady-state natural convection. *International Journal of Heat and Mass Transfer*, 216:124593, 2023. doi: 10.1016/j.ijheatmasstransfer.2023.124593.

[10] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi: 10.1038/s42256-021-00302-5.

[11] Sifan Wang, Hanwen Wang, and Paris Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Science Advances*, 7 (40):eabi8605, 2021. doi: 10.1126/sciadv.abi8605.

[12] Xiaoyu Zhao, Xiaoqian Chen, Zhiqiang Gong, Weien Zhou, Wen Yao, and Yunyang Zhang. RecFNO: A resolution-invariant flow and heat field reconstruction method from sparse observations via Fourier neural operator. *International Journal of Thermal Sciences*, 195:108619, 2023. doi: 10.1016/j.ijthermalsci.2023.108619.

[13] X. F. Yu, Wenxiang Gao, Kailun Guo, Z. Zhang, Heng Wang, Mingjun Wang, Suizheng Qiu, and Wenxi Tian. Fourier neural operator-based approach for real-time prediction of high-temperature heat pipes frozen startup. *Applied Thermal Engineering*, 302:131964, 2026. doi: 10.1016/j.applthermaleng.2026.131964.

[14] İzzet Şahin, Christian Moya, Amirhossein Mollaali, Guang Lin, and Guillermo Paniagua. Deep operator learning-based surrogate models with uncertainty quantification for optimizing internal cooling channel rib profiles. *International Journal of Heat and Mass Transfer*, 219: 124813, 2023. doi: 10.1016/j.ijheatmasstransfer.2023.124813.

[15] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[16] Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks for heat transfer problems. *ASME Journal of Heat and Mass Transfer*, 143(6):060801, 2021. doi: 10.1115/1.4050542.

[17] Darioush Jalili, Seohee Jang, Mohammad Jadidi, Giovanni Giustini, Amir Keshmiri, and Yasser Mahmoudi. Physics-informed neural networks for heat transfer prediction in two-phase flows. *International Journal of Heat and Mass Transfer*, 221:125089, 2023. doi: 10.1016/j.ijheatmasstransfer.2023.125089.

[18] Tim De Ryck and Siddhartha Mishra. Numerical analysis of physics-informed neural networks and related models in physics-informed machine learning. *Acta Numerica*, 33:633–713, 2024. doi: 10.1017/S0962492923000089.

[19] Kuang Luo, Jingshang Zhao, Ying-Ping Wang, Jiayao Li, Junjie Wen, Jiong Liang, Henry Soekmadji, and Shaolin Liao. Physics-informed neural networks for PDE problems: A comprehensive review. *Artificial Intelligence Review*, 58(10):323, 2025. doi: 10.1007/s10462-025-11322-7.

[20] Prashant K. Jha. Residual-based error corrector operator to enhance accuracy and reliability of neural operator surrogates of nonlinear variational boundary-value problems. *Computer Methods in Applied Mechanics and Engineering*, 419:116595, 2023. doi: 10.1016/j.cma.2023.116595.

[21] Lianghao Cao, Thomas O'Leary-Roseberry, Prashant K. Jha, John Tinsley Oden, and Omar N. Ghattas. Residual-based error correction for neural operator accelerated infinite-dimensional Bayesian inverse problems. *Journal of Computational Physics*, 486:112104, 2023. doi: 10.1016/j.jcp.2023.112104.

[22] Min Zhu, Handi Zhang, Anran Jiao, George Em Karniadakis, and Lu Lu. Reliable extrapolation of deep neural operators informed by physics or sparse observations. *Computer Methods in Applied Mechanics and Engineering*, 412:116064, 2023. doi: 10.1016/j.cma.2023.116064.

[23] Somdatta Goswami, Katiana Kontolati, Michael D. Shields, and George Em Karniadakis. Deep transfer operator learning for partial differential equations under conditional shift. *Nature Machine Intelligence*, 4(12):1155–1164, 2022. doi: 10.1038/s42256-022-00569-2.

[24] Paolo Conti, Mengwu Guo, Andrea Manzoni, and Jan S. Hesthaven. Multi-fidelity surrogate modeling using long short-term memory networks. *Computer Methods in Applied Mechanics and Engineering*, 404:115811, 2022. doi: 10.1016/j.cma.2022.115811.

[25] Wenqian Chen and Panos Stinis. Feature-adjacent multi-fidelity physics-informed machine learning for partial differential equations. *Journal of Computational Physics*, 498:112683, 2023. doi: 10.1016/j.jcp.2023.112683.

[26] Yanfang Lyu, Xiaoyu Zhao, Zhiqiang Gong, Xiao Jun Kang, and Wen Yao. Multi-fidelity prediction of fluid flow based on transfer learning using Fourier neural operator. *Physics of Fluids*, 35(7):077118, 2023. doi: 10.1063/5.0155555.

[27] Guenter Ahlers, Siegfried Großmann, and Detlef Lohse. Heat transfer and large scale dynamics in turbulent Rayleigh–Bénard convection. *Reviews of Modern Physics*, 81(2):503–537, 2009. doi: 10.1103/RevModPhys.81.503.

[28] Hiufai Yik, Valentina Valori, and Stephan Weiss. Turbulent Rayleigh–Bénard convection under strong non-Oberbeck–Boussinesq conditions. *Physical Review Fluids*, 5(10):103502, 2020. doi: 10.1103/PhysRevFluids.5.103502.

[29] Michal Macek, Georgy Zinchenko, Věra Musilová, P. Urban, and Jörg Schumacher. Assessing non-Oberbeck–Boussinesq effects of convection in cryogenic helium. *Physical Review Fluids*, 8(9):094606, 2023. doi: 10.1103/PhysRevFluids.8.094606.

[30] Xiaomin Pan and Jung-Il Choi. Non-Oberbeck–Boussinesq effects on a water-filled differentially heated vertical cavity. *Physics of Fluids*, 35(11):113609, 2023. doi: 10.1063/5.0173049.

[31] Jinsong Zhang, Yongyong Wu, Nan Gui, Xingtuan Yang, Jiyuan Tu, and Shengyao Jiang. Investigation of compressible flow under low Mach number in an enclosed square cavity with a novel non-Boussinesq algorithm. *Physics of Fluids*, 35(12):123120, 2023. doi: 10.1063/5.0175738.

[32] Hidemoto Satake and Toshio Tagawa. Mathematical modeling for low-Mach-number compressible thermal convection analysis in a square enclosure. *Physics of Fluids*, 36(12):127168, 2024. doi: 10.1063/5.0243786.

[33] Peter Bella, Eduard Feireisl, and Florian Oschmann. Rigorous derivation of the Oberbeck–Boussinesq approximation revealing unexpected term. *Communications in Mathematical Physics*, 403(3):1245–1273, 2023. doi: 10.1007/s00220-023-04823-5.